

CS F407 - Week 1 Lab: Neural Models (XOR, Depth, Activations, Output Layers)

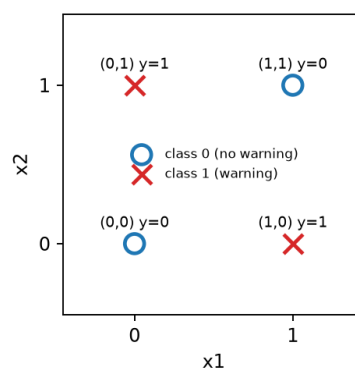
Advay Matapurkar | ID: 2023A7PS0459G

Code: `xor_lab.py` (all experiments; seed 0, float64, CPU). Run with `python xor_lab.py`; every number below is printed by that script.

1. Task 1 - Problem specification

Input space $X = \{0,1\}^2$; output space $Y = \{0,1\}$ (1 = sensor-disagreement warning). Labelled examples: (0,0)→0, (0,1)→1, (1,0)→1, (1,1)→0.

Sketch: in the (x_1, x_2) plane the class-0 points sit at opposite corners (0,0),(1,1) and the class-1 points at the other two corners (0,1),(1,0) (sketch below).



Not linearly separable: a line $w \cdot x + b = 0$ would need $w \cdot x + b < 0$ at (0,0) and (1,1), and > 0 at (0,1) and (1,0). Summing the first pair gives $w_1 + w_2 + 2b < 0$, summing the second gives $w_1 + w_2 + 2b > 0$: a contradiction. (Equivalently, the segments joining same-class points cross at (0.5, 0.5), so the convex hulls intersect.)

Prediction for affine+sigmoid: it cannot classify all four points correctly (one line can get at most 3 of 4 right); minimising BCE should settle at $\ln 2 = 0.693$ with all outputs near 0.5. **Result:** final loss 0.6931, probabilities [0.5, 0.5, 0.5, 0.5], so the prediction is confirmed (every output is 0.5, so the thresholded labels are arbitrary; the rule $p > 0.5$ gave all zeros, 2/4 correct). *Think about it:* XOR tests the claim that what matters is the *kind* of representation (a nonlinear feature map), not the parameter count.

2. Task 2 - Model design and validation criteria

Design: 2 inputs → 2 hidden units (tanh; sigmoid and ReLU later) → 1 output logit with sigmoid output and binary cross-entropy (implemented as BCEWithLogitsLoss); full-batch Adam, lr 0.05, 3000 steps.

Why a hidden nonlinearity: without it, $W_2(W_1x + b_1) + b_2$ collapses to one affine map, which cannot represent XOR. The nonlinearity lets the two hidden units cut the plane with different half-spaces (roughly OR and AND) that the output combines. **Why sigmoid+BCE:** the target is one yes/no; sigmoid gives a probability, BCE is the Bernoulli negative log-likelihood, and the logit gradient is simply $\text{sigmoid}(z) - y$, with no saturation-induced vanishing when the output is confidently wrong. **Hidden targets:** hidden units are given none; backpropagation delivers dL/dh through the chain rule, so the output loss determines what each unit computes.

Success criteria: (1) final loss far below $\ln 2$ (e.g. < 0.01); (2) all four thresholded predictions correct with probabilities well away from 0.5; (3) first-layer gradient non-zero early *and* the loss actually decreasing; (4) autograd gradient agrees with finite differences; (5) repeated-run behaviour over several seeds.

3. Task 3 - LLM prompt, generated code, corrections

Prompt. After completing Tasks 1 and 2, I wrote the following prompt and gave it to the LLM (Claude, via Claude Code). It is reproduced verbatim; only the line breaks are adjusted to fit the page. The follow-up prompt for the three-class change (Task 5) is in section 5.

Generate minimal PyTorch code for the following model and dataset. Do not change the architecture or task.

Dataset (XOR, full batch): (0,0)->0, (0,1)->1, (1,0)->1, (1,1)->0.

Model: a 2-2-1 network (2 inputs, 2 hidden units, 1 output) with tanh hidden activation and a single output logit trained with BCEWithLogitsLoss (i.e. a sigmoid binary output). Use random weight initialisation.

Training: full-batch training for a few thousand lightweight CPU steps.

After training, print the initial and final loss, all four probabilities, the thresholded labels, and at least one parameter-gradient tensor (the first-layer weight gradient) after backward(). Set a random seed for reproducibility and explain each test in one sentence.

Inspection: forward pass and scalar loss: `loss = loss_fn(net(X), y)` in `train()`; reverse-mode AD: `loss.backward()`; parameter update: `opt.step()`. **Changes to the first draft:** (i) float64, so finite-difference checks are accurate to about 1e-10; (ii) multi-seed runs and a post-training ReLU diagnostic, because the single-seed ReLU failure needed explaining. The assistant's first draft also contained a syntax slip (an unescaped newline in a print string), which was caught and fixed on first execution. *Verifiable without running:* architecture, loss/activation pairing, where backward/step occur, dataset. *Needs execution:* convergence, gradient correctness, seed sensitivity.

4. Task 4 - Results

Part A (tanh, seed 0). Initial loss 0.7102, final loss 0.000108. Probabilities [0.0001, 0.9998, 0.9999, 0.0001]; labels [0.0, 1.0, 1.0, 0.0]; all four correct: True.

Part B (backprop check). After `backward()`, `net[0].weight.grad` holds $dL/dW(1)$: entry $[i,j]$ is the partial derivative of the scalar loss with respect to $W1[i,j]$ at the current parameters. With a mean loss $L = (1/4) \sum_n L_n$, linearity of differentiation gives $dL/dW = (1/4) \sum_n dL_n/dW$, the average of the per-example gradients (verified numerically, max difference 0.0e+00). Both checks were run at initialisation (seed 0), where the gradient is not yet close to zero (norm 0.0437). Autograd vs central finite differences: max error 3.5e-11, i.e. 1e-09 relative to the largest entry, so the gradient is correct.

Part C (symmetry). With all parameters zero, the two rows of $W(1)$ stayed exactly identical at steps 0, 1, 10, 100, 1000 and 2999 (max difference 0.0). The loss stayed at 0.6931 with all probabilities 0.5: since $W2=0$ the hidden-layer gradient is also zero, so only the output bias learns. To rule out 'merely zero gradient', I also used one identical non-zero constant (0.5) for all parameters. Gradients are then non-zero and $W1$ moved to about -7.77 in every entry, yet the rows stayed exactly equal and the loss stalled at 0.4774 (probabilities [0.0, 0.667, 0.667, 0.667]). Reason: identical rows give $h1=h2$, so with equal outgoing weights they receive the same backpropagated gradient, and the update preserves equality by induction. The network is effectively one hidden unit, a function of $x1+x2$ only, which cannot express XOR.

Part D - activation experiment (identical initial weights, seed 0)

Hidden act.	Final loss	4/4 correct?	Early $\ grad\ W1\ _2$ (step 0)	Solved, seeds 0-19	Mean step-0 norm (20 seeds)
sigmoid	0.000284	Yes	0.0096	7/20	0.0083
tanh	0.000108	Yes	0.0437	10/20	0.0305
relu	0.693147	No	0.0412	6/20	0.0345

Interpretation. On seed 0, tanh and sigmoid solved XOR and ReLU did not. Sigmoid's early gradient norm (0.0096) is about 4.5x smaller than tanh's (0.0437): sigmoid' ≤ 0.25 and its outputs are not zero-centred, so backpropagated signals are scaled down; it still converged, only from a weaker initial signal. ReLU had a large early gradient (0.0412) yet ended at loss $\ln 2$. The post-training diagnostic shows both hidden units have $z > 0$ for all four inputs: they are not dead, but they operate in the linear regime, so the network is affine and cannot represent XOR. A large gradient is not a useful one if the function it moves within cannot express the target.

Across 20 seeds all three activations fail on many initialisations (solved: sigmoid 7/20, tanh 10/20, ReLU 6/20 with this 2-unit bottleneck), so four points cannot rank activations; the seed-0 ordering is largely initialisation luck, and only the smaller sigmoid gradient scale is consistent across seeds.

Think about it (telling small gradients apart): a saturated sigmoid/tanh unit has large $|z|$ with activation near its asymptote, so $f'(z)$ is about 0 for the inputs it sees; a negative ReLU has $z < 0$ and activation exactly 0. Inspect z and h per unit per input: saturation shows as large $|z|$ with h near 0/1 (or +1); a dead ReLU shows as $z < 0$ on every input, so that unit's incoming-weight gradient is exactly 0. At initialisation the sigmoid derivatives were 0.18-0.25 (unsaturated); after training they were at most 0.0077 (saturated, as expected for confident outputs).

5. Task 5 - Three-class extension

Prompt. For this task I gave the LLM the follow-up prompt below (verbatim; only the line breaks are adjusted to fit the page).

Modify only the output/loss portion of the PyTorch code you gave me for the 2-2-1 XOR network. Do not change the input, the hidden layer, the hidden activation, the optimiser, the seed, or the training loop structure.

New task: convert the same two-sensor input into a three-class decision.

Class 0: both sensors inactive, (0,0)

Class 1: sensors disagree, (0,1) or (1,0)

Class 2: both sensors active, (1,1)

Replace the single output logit with three logits and train with multiclass cross-entropy (`nn.CrossEntropyLoss` with integer class labels). Keep the hidden representation small.

After training, print: the shape of the final weight matrix, the number of logits per example, the final loss, and the softmax class probabilities for all four inputs. For one example, print the softmax probability vector and verify numerically that its components sum to approximately 1. Also verify numerically that the gradient of the loss with respect to the logits equals $p - y$ (account for the $1/N$ factor from the mean over examples).

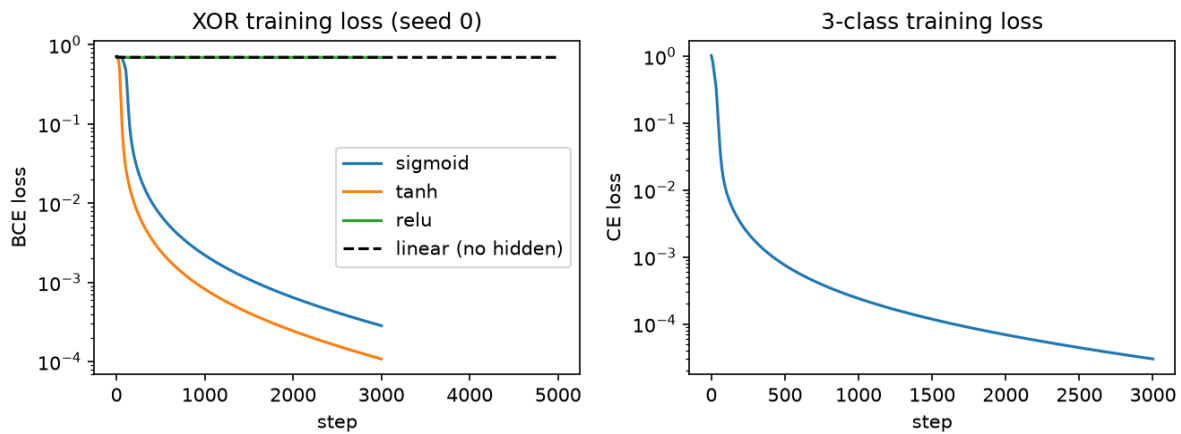
Optional diagnostic: add the same constant (for example 100) to all three logits before softmax and check that the probability vector is unchanged apart from floating-point roundoff. Then show that a naive softmax overflows on large logits (for example [1000, 1001, 1002]) and that subtracting the maximum logit first fixes it. Explain each test in one sentence.

Only the output/loss changed: the same 2 tanh hidden units, with the single output replaced by 3 output logits and the loss by `nn.CrossEntropyLoss` with integer labels 0/1/2 for (0,0) / disagree / (1,1). **Predictions before running:** final weight matrix shape (3 outputs x 2 hidden) = (3, 2); 3 logits per example; softmax outputs $\exp(z_k)/\sum_j \exp(z_j)$ are non-negative and divided by their own sum, so they sum to 1; for $L = -\sum_k y_k \log p_k$ with $p = \text{softmax}(z)$, $dL/dz_k = p_k - y_k$ (using $dp_j/dz_k = p_j(\delta_{jk} - p_k)$ and $\sum y = 1$). With a mean over $N=4$ examples the gradient carries an extra $1/N$ (verified, max error $3.4e-21$).

input	p(class 0)	p(class 1)	p(class 2)	pred
(0, 0)	0.999970	0.000030	0.000000	0
(0, 1)	0.000015	0.999972	0.000013	1
(1, 0)	0.000013	0.999976	0.000011	1
(1, 1)	0.000000	0.000038	0.999962	2

Final loss $3.00e-05$; all four predictions correct. For input (0,1) the vector is [1.53e-05, 0.999972, 1.28e-05] and its components sum to 1.000000000000. **Optional diagnostic:** adding 100 to all logits changes the probabilities by at most $1.1e-19$ (round-off), since $\exp(z+c)/\sum \exp(z+c)$ cancels e^c . Stable implementations subtract $\max(z)$ so every exponent is ≤ 0 and cannot overflow: naive exp on [1000,1001,1002] gives [nan, nan, nan], whereas subtracting the max gives [0.090, 0.245, 0.665].

Think about it (vocabulary scale): unchanged: softmax, cross-entropy, the $p-y$ logit gradient, log-sum-exp stabilisation, averaging over examples. Changing dramatically: the output layer (tens of thousands of logits dominate cost and memory), and everything producing the hidden representation (deep transformers over long contexts instead of a 2-unit tanh layer), plus data and compute scale.



6. Reflection questions

1. **Depth vs nonlinearity.** Stacking affine layers changes nothing, since they collapse to one affine map. Adding tanh between them let a 2-2-1 net solve XOR, while affine+sigmoid stayed at loss $\ln 2$. The ReLU run also showed that the nonlinearity must actually be *used*: its units stayed in the linear regime and it failed.

2. **Useful signal, not merely non-zero.** The loss fell from 0.71 to $1e-4$ and predictions moved to the correct corners; the gradient matched finite differences to a relative error of about $1e-9$, so it is the true derivative. Also, the ReLU run had a larger gradient norm than sigmoid yet learned nothing, and the constant-init run had non-zero gradients yet stalled at loss 0.477, so a non-zero gradient alone is not evidence.

3. **Symmetric initialisation.** Identical hidden units compute identical outputs, receive identical backpropagated gradients and therefore identical updates, so equality persists at every step. Zero init is the extreme case (hidden gradients are also 0 because $W_2=0$). Random initialisation breaks the symmetry.

4. **Activation and gradient.** Engineering observation: at step 0 (same initial weights) sigmoid gave a gradient norm about 4.5x smaller than tanh (0.0096 vs 0.0437); ReLU's was 0.0412. Seed 0 solved XOR for sigmoid and tanh but not ReLU; over 20 seeds the success counts were 7, 10 and 6. Scientific explanation: sigmoid' ≤ 0.25 and is not zero-centred; tanh' ≤ 1 is zero-centred; ReLU' is 0 or 1 and yields a piecewise-linear function whose pieces may all coincide, as observed. Four points and one small architecture cannot rank activations universally.

5. **Output and loss together.** The loss should be the likelihood matching the output distribution: sigmoid+BCE for a Bernoulli target, softmax+CE for a categorical one. This gives the clean logit gradient $p-y$ and a correct probabilistic reading. A mismatch (e.g. sigmoid with squared error, or independent sigmoids for mutually exclusive classes) optimises something else and can learn slowly through saturation.

6. **LLM help vs verification.** Productivity: quickly producing the training loop, per-example-gradient comparison, finite-difference check and multi-seed sweep. Human verification was essential: the first script had a syntax slip, and the ReLU seed-0 failure could have been misread as 'ReLU is bad' or 'dead ReLU'; only inspecting pre-activations showed the units were active everywhere and the network effectively affine. The $1/N$ factor in $p-y$ also had to be checked, not assumed.

7. **Scaling up.** Keep: loss curves, held-out accuracy, gradient-norm and NaN monitoring, per-layer activation statistics, the single-batch overfit test, seed repeats, and finite-difference checks on a few random parameters of a small copy. Too expensive at scale: exhaustive finite-difference checks (two forward passes per parameter), exhaustive multi-seed sweeps, and full per-example gradient comparison.